

机器无关优化完整模拟卷 06（题目与完整解析）

卷面说明

考试时间：120 分钟

总分：100 分

说明：本卷独立覆盖局部优化、到达定值、活跃变量、可用表达式、复制传播、支配与自然循环。

所有 CFG、gen/kill、use/def 和表达式全集均在题面中给出。

A 卷题目

一、语义不变优化与数据流框架（20 分）

给定基本块：

```
(1) t1 = a + b
(2) t2 = a + b
(3) x  = t2
(4) y  = x
(5) z  = 3 * 4
```

块出口只有 x 活跃。

1. 指出公共子表达式并消除。
2. 对复制语句做传播。
3. 对常量表达式做折叠。
4. 删除死代码，写出最终基本块。
5. 说明正向/逆向数据流分析中 IN、OUT、transfer 和 meet 的作用。

二、到达定值分析（20 分）

CFG：

```
Entry -> B1
B1 -> B2
B2 -> B3
B2 -> B4
B3 -> B4
B4 -> B2
```

给定：

```
gen[B1] = {d1, d2, d3}    kill[B1] = {d4, d5, d6, d7}
gen[B2] = {d4, d5}        kill[B2] = {d1, d2, d7}
gen[B3] = {d6}             kill[B3] = {d3}
gen[B4] = {d7}             kill[B4] = {d1, d4}
```

1. 写出分析方向、meet 和 transfer 方程。
2. 从所有 OUT 为空集开始，采用就地更新并按 B1、B2、B3、B4 顺序处理，写出第一轮 OUT。
3. 继续迭代到不动点，写出所有 IN/OUT。
4. 说明为什么该分析使用并集而不是交集。
5. 写出 worklist 算法的停止条件。

三、活跃变量分析（20 分）

CFG：

```
B1 -> B2
B2 -> B3
B2 -> B4
B3 -> B2
B4 -> Exit
```

基本块集合：

use[B1]={}	def[B1]={x, y}
use[B2]={x, y}	def[B2]={z}
use[B3]={z}	def[B3]={x}
use[B4]={z}	def[B4]={}

1. 写出活跃变量的定义。
2. 写出方向、meet 和 transfer 方程。
3. 从所有 IN/OUT 为空集开始迭代，写出最终 IN/OUT。
4. 根据结果判断 x、y 在 B3 出口是否活跃。
5. 说明活跃变量分析在寄存器分配和死代码删除中的用途。

四、可用表达式与复制传播（20 分）

表达式全集：

$U = \{a+b, c*d\}$

CFG 与局部集合：

```
Entry -> B1
B1 -> B2
B1 -> B3
B2 -> B4
B3 -> B4

e_gen[B1]={a+b}  e_kill[B1]={}
e_gen[B2]={c*d}  e_kill[B2]={a+b}
e_gen[B3]={a+b}  e_kill[B3]={}
e_gen[B4]={}     e_kill[B4]={}
```

1. 写出可用表达式定义。
2. 写出分析方向、meet、边界条件和 transfer 方程。
3. 计算所有基本块最终 IN/OUT。
4. 判断进入 B4 时 a+b 和 c*d 是否可用。
5. 写出在使用点把 x 替换为 y 所需的复制传播条件。

五、支配、后支配、回边与自然循环（20 分）

给定 CFG：

```
Entry -> B1
B1 -> B2
B2 -> B3
B2 -> B4
B3 -> B2
B4 -> Exit
```

1. 计算所有结点的 Dom 集和 idom。
2. 写出支配树。
3. 计算所有结点的 PostDom 集和 ipdom。
4. 找出全部回边并构造自然循环。
5. 判断该 CFG 是否可归约，并写出依据。

B 按考试标准重写答案

一、语义不变优化与数据流框架答案

1. 公共子表达式消除

基本块中：

```
(1) t1 = a + b
(2) t2 = a + b
```

从 (1) 到 (2) 之间没有对 a 或 b 的重新定义，因此 (2) 的 $a+b$ 与 (1) 是同一个公共子表达式。

改写：

```
(1) t1 = a + b
(2) t2 = t1
```

2. 复制传播

继续使用复制关系：

```
t2 = t1
x = t2
y = x
```

传播后：

```
x = t1
y = t1
```

3. 常量折叠

```
z = 3 * 4
```

编译期即可计算：

```
z = 12
```

4. 删除死代码并写最终基本块

块出口只有 x 活跃。 $t2$ 、 y 、 z 的值在出口不活跃，并且这些赋值没有副作用，因此可以删除。

最终基本块：

```
t1 = a + b
x = t1
```

优化过程表：

原语句	优化依据	处理结果
$t1 = a + b$	需要给 x 使用	保留
$t2 = a + b$	与 (1) 是公共子表达式	变为 $t2=t1$ 后删除
$x = t2$	复制传播	改为 $x=t1$ 并保留
$y = x$	y 出口不活跃	删除
$z = 3 * 4$	常量折叠为 $z=12$ ，但 z 不活跃	删除

5. 数据流框架术语

概念	作用
IN[B]	进入基本块 B 前的数据流事实
OUT[B]	离开基本块 B 后的数据流事实
transfer	描述基本块如何改变数据流事实
meet	多条控制流路径汇合时合并事实

方向区别：

分析类型	传播方向	典型方程
正向分析	从前驱到后继	$IN[B] = meet \ OUT[pred]$
逆向分析	从后继到前驱	$OUT[B] = meet \ IN[succ]$

meet 使用并集还是交集，取决于问题是 may 分析还是 must 分析。

二、到达定值分析答案

1. 方向、meet、transfer

到达定值定义：若某个定值 d 存在一条路径到达程序点，且路径上没有被同变量的新定值杀死，则 d 到达该点。

因此它是正向 may 分析：

```

IN[B] = union OUT[P], P 是 B 的所有前驱
OUT[B] = gen[B] union (IN[B] - kill[B])
OUT[Entry] = {}

```

2. 第一轮 OUT

从所有 OUT 为空集开始，并按 B1、B2、B3、B4 就地更新。

块	IN 第一轮	OUT 第一轮计算	OUT 第一轮
B1	{}	$\{d1, d2, d3\} \cup ({} - kill[B1])$	$\{d1, d2, d3\}$
B2	$OUT[B1] \cup OUT[B4] = \{d1, d2, d3\}$	$\{d4, d5\} \cup (\{d1, d2, d3\} - \{d1, d2, d7\})$	$\{d3, d4, d5\}$
B3	$OUT[B2] = \{d3, d4, d5\}$	$\{d6\} \cup (\{d3, d4, d5\} - \{d3\})$	$\{d4, d5, d6\}$
B4	$OUT[B2] \cup OUT[B3] = \{d3, d4, d5, d6\}$	$\{d7\} \cup (\{d3, d4, d5, d6\} - \{d1, d4\})$	$\{d3, d5, d6, d7\}$

3. 不动点 IN/OUT

继续迭代直到集合不再变化，结果为：

基本块	IN	OUT
B1	{}	$\{d1, d2, d3\}$
B2	$\{d1, d2, d3, d5, d6, d7\}$	$\{d3, d4, d5, d6\}$
B3	$\{d3, d4, d5, d6\}$	$\{d4, d5, d6\}$
B4	$\{d3, d4, d5, d6\}$	$\{d3, d5, d6, d7\}$

关键点在 B2：B2 的前驱是 B1 和 B4，循环回边 B4 → B2 会把 {d5, d6, d7} 带回 B2。

4. 为什么使用并集

到达定值是 may 分析。“可能到达”只要求存在一条路径。因此只要任一前驱能带来某个定值，该定值就应保留。

```
meet = union
```

若使用交集，会错误地只保留所有路径都带来的定值，这不符合“可能到达”的定义。

5. worklist 停止条件

worklist 算法反复处理受影响基本块。停止条件是：

工作队列为空，且所有 IN/OUT 集合在最近一次传播中没有变化。

这时已经达到不动点。

三、活跃变量分析答案

1. 活跃变量定义

变量 v 在程序点 p 活跃，当且仅当：

存在一条从 p 出发的路径，在 v 被重新定义之前使用 v 。

也就是说，当前值将来还可能被用到。

2. 方向、meet、transfer

活跃变量分析关心“将来是否会使用”，所以是逆向 may 分析：

$$\begin{aligned} \text{OUT}[B] &= \text{union } \text{IN}[S], S \text{ 是 } B \text{ 的所有后继} \\ \text{IN}[B] &= \text{use}[B] \text{ union } (\text{OUT}[B] - \text{def}[B]) \end{aligned}$$

出口边界：

$$\text{OUT}[B4] = \{\}$$

3. 最终 IN/OUT

基本块	use	def	IN	OUT
B1	{}	{x, y}	{}	{x, y}
B2	{x, y}	{z}	{x, y}	{y, z}
B3	{z}	{x}	{y, z}	{x, y}
B4	{z}	{}	{z}	{}

推导例子：

$$\begin{aligned} \text{OUT}[B3] &= \text{IN}[B2] = \{x, y\} \\ \text{IN}[B3] &= \text{use}[B3] \text{ union } (\text{OUT}[B3] - \text{def}[B3]) \\ &= \{z\} \text{ union } (\{x, y\} - \{x\}) \\ &= \{y, z\} \end{aligned}$$

4. B3 出口处 x 、 y 是否活跃

B3 的出口集合是：

$$\text{OUT}[B3] = \{x, y\}$$

因此：

变量	是否活跃	原因
x	是	B3 后回到 B2, B2 使用 x
y	是	B3 后回到 B2, B2 使用 y, 且 B3 不定义 y

5. 用途

用途	说明
寄存器分配	不活跃变量占用的寄存器可优先覆盖；活跃变量需要保留或 spill
死代码删除	若赋值结果之后不活跃且语句无副作用，则该赋值可删除
指令选择	可根据下一次使用信息减少保存和重载

四、可用表达式与复制传播答案

1. 可用表达式定义

表达式 e 在程序点 p 可用，当且仅当：

从 Entry 到 p 的每条路径都已经计算过 e ，
并且从最后一次计算 e 到 p 之间没有修改 e 的任何操作数。

关键词是“每条路径”，所以这是 must 分析。

2. 方向、meet、边界、transfer

可用表达式是正向 must 分析：

```
OUT[Entry] = {}
IN[B] = intersection OUT[P], P 是 B 的所有前驱
OUT[B] = e_gen[B] union (IN[B] - e_kill[B])
```

迭代初值：除 Entry 外，其余块的 OUT 可初始化为全集：

```
U = {a+b, c*d}
```

3. 最终 IN/OUT

基本块	IN	OUT
B1	{}	{a+b}
B2	{a+b}	{c*d}
B3	{a+b}	{a+b}
B4	{}	{}

B4 的关键计算：

```
IN[B4] = OUT[B2] intersection OUT[B3]
        = {c*d} intersection {a+b}
        = {}
```

4. 进入 B4 时表达式是否可用

表达式	是否可用	原因
a+b	否	经 B2 的路径上 B2 kill 了 a+b
c*d	否	经 B3 的路径没有计算 c*d

因此进入 B4 时两个表达式都不可用。

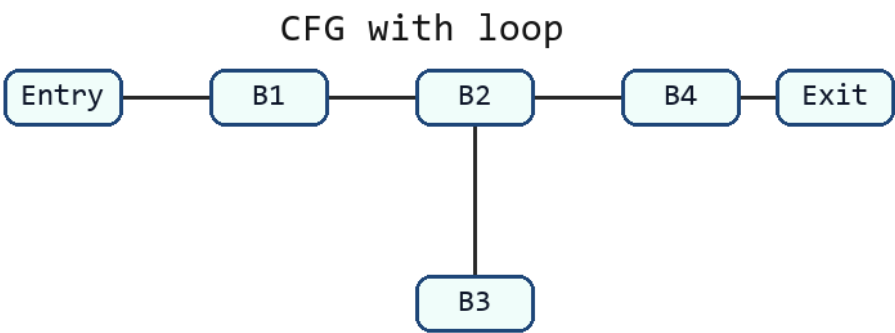
5. 复制传播条件

要在使用点 u 把 x 替换为 y ，必须保证 x 当前值确实来自同一个 y ：

条件	说明
复制 $x=y$ 到达使用点	从 Entry 到 u 的每条路径上，都有同一个可用复制关系

条件	说明
x 未被重定义	x=y 之后到 u 之间不能重新定义 x
y 未被重定义	否则当前 y 与复制时的 y 不一定相同
类型和别名安全	替换不能改变程序语义

五、支配、后支配、回边与自然循环答案



CFG with loop

1. Dom 集和 idom

支配定义：若从 Entry 到结点 n 的每条路径都经过 d，则 d 支配 n。

Dom 集：

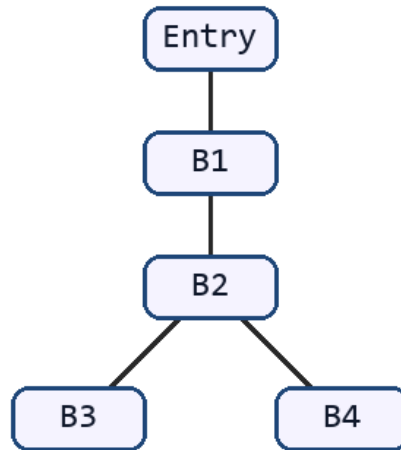
结点	Dom
Entry	{Entry}
B1	{Entry, B1}
B2	{Entry, B1, B2}
B3	{Entry, B1, B2, B3}
B4	{Entry, B1, B2, B4}
Exit	{Entry, B1, B2, B4, Exit}

直接支配者：

结点	idom
B1	Entry
B2	B1
B3	B2
B4	B2
Exit	B4

2. 支配树

Dominator tree



支配树

Entry
B1
B2
B3
B4
Exit

3. PostDom 集和 ipdom

后支配定义：若从结点 n 到 $Exit$ 的每条路径都经过 p ，则 p 后支配 n 。

结点	PostDom
Exit	{Exit}
B4	{B4, Exit}
B3	{B3, B2, B4, Exit}
B2	{B2, B4, Exit}
B1	{B1, B2, B4, Exit}
Entry	{Entry, B1, B2, B4, Exit}

直接后支配者：

结点	ipdom
Entry	B1
B1	B2
B2	B4
B3	B2
B4	Exit

4. 回边和自然循环

检查 CFG 中指向前面循环头的边：

B3 $\rightarrow$ B2

因为 B2 支配 B3，所以 B3 $\rightarrow$ B2 是回边。

自然循环构造：

从回边尾 B3 开始，反向沿前驱搜索，直到回边头 B2。
包含回边头 B2 和搜索到的所有结点。

结果：

自然循环 = {B2, B3}

5. 是否可归约

该 CFG 可归约。依据：

循环入口唯一，为 B2。

回边 B3 → B2 的头 B2 支配尾 B3。

不存在多个入口进入同一个循环体的结构。